

BUDGER

Weekly Category Drill-down

Updated implementation plan

Locked requirements from the supplied support screenshots

Implementation status

Planning is complete. No production code changes are included in this document. When implementation begins, every shipping change belongs in **export/frontend/** or **export/backend/**; artifacts remain for local development and verification only.

Scope

Add a long-press entry point to the dashboard category budget donut and open a second, category-specific donut showing the selected month's spending across four calendar periods. The feature must feel like the existing household and personal donut interactions, use authenticated server data, and hand off to the existing full-month transaction list without inventing a parallel flow.

Non-goals

- Do not change the existing short-tap category selection.
- Do not change center double-tap mode switching.
- Do not make uncategorized, recurring-payment, unbudgeted, or empty categories drillable.
- Do not download all transactions to the browser to calculate the weekly chart.
- Do not deploy to Vercel or Render from this work; GitHub push is the handoff point.

Interaction entry point

Reuse the household donut's pointer-based long-press model. The category segment must be eligible before a timer starts, and the normal tap action must remain the same when the hold does not fire.

- Start a roughly 500 ms hold on an eligible dashboard category segment.
- Cancel on pointer release, pointer cancellation, or movement away from the segment.
- When the hold fires, suppress the click/tap selection that would otherwise follow.
- Preserve the current short-tap behavior and center double-tap mode switching unchanged.
- Eligibility requires a real category with a positive budget and spending in the selected month.
- No drill-down for uncategorized spending, recurring-payment slices, categories without a positive budget, or categories with no spending.

Weekly state

- Dashboard owns the selected drill-down category and the weekly state.
- The existing view remains the category budget donut; long press transitions to the selected category's weekly donut.
- The weekly header includes a back arrow, category name, selected month, and effective category budget.
- The selected month is the month currently shown by the dashboard.
- Back navigation returns to the original category donut and clears weekly state completely.
- Copy the existing household/personal donut animation sequence 1:1: fade other segments, expand the selected segment into a neutral arc, then reveal the weekly chart.

Important handoff rule

The weekly screen must include a **Show transactions** action. It returns the user to the home/spending tab at /, requests the full selected month, and applies the existing category-name filter. It must not filter only the first 15 records. Clearing the filter must restore the normal unfiltered list. Returning to the dashboard later must show the default dashboard view, not the weekly drill-down.

Files touched by the feature

Area	Shipping location	Responsibility
Donut interaction	export/frontend/src/components/DonutBudgetChart.tsx	Long press, eligibility, click suppression, animation reuse
Weekly chart	export/frontend/src/components/WeeklyCategoryDonut.tsx	Four-period donut, legend, exact percentage math
Dashboard state	export/frontend/src/pages/Dashboard.tsx	Selected category/month state, query, transition, back/reset, handoff
Copy	export/frontend/src/lib/i18n.ts	Week labels, date ranges, back/help text, transaction CTA
Summary API	export/backend/src/routes/summary.ts	Authenticated weekly category aggregation
Contract	lib/api-spec/openapi.yaml + generated client/types	OpenAPI source and regenerated hooks/schemas; sync shipping copies as applicable

Endpoint

Use the dedicated summary route rather than downloading transactions to the browser. The workspace already contains the draft **GET /summary/category-weeks** contract and generated types; implementation work must audit, complete, and synchronize it rather than create a second endpoint.

Request	Required behavior
GET /api/summary/category-weeks	Authenticated request for one category and one dashboard month.
month=YYYY-MM	Validate the exact month format; use the dashboard's selected month.
categoryId=<positive integer>	Validate the category identifier and enforce ownership.
currency=<display currency>	Apply the same native/display currency rules as the existing dashboard summary.

Response shape

Return the category context and exactly four calendar periods. The server owns the date boundaries so the frontend cannot duplicate or drift from the aggregation rules.

```
month, categoryId, categoryName, budget, totalSpent, weeks[4]
week: { index, startDate, endDate, days, spent, entriesCount }
```

Server-side inclusion/exclusion rules

- Current-user ownership and selected-category ownership are checked server-side.
- Use the selected month and the request's native/display currency filtering rules.
- Exclude locked or unavailable currency entries.
- Exclude realized-goal-funded entries.
- Exclude Larder funds.
- Use the same effective category budget as the existing dashboard donut, including applicable budget-stretch adjustments.
- Return actual ISO date-only startDate and endDate values; the frontend formats them for display.
- Use the same transaction inclusion semantics as the existing dashboard summary so weekly totals reconcile to totalSpent.

Security and consistency

Unauthorized, invalid, or unavailable category requests must be rejected. The route must not expose weekly data for another user's category. The response should be sufficient for rendering the chart without a second client-side ownership or date-boundary implementation.

Deterministic calendar periods

Split the actual calendar month into four contiguous periods. Extra days are assigned from week 1 forward. The API returns the actual boundaries and day counts.

Month length	Period day counts
28 days	7 / 7 / 7 / 7
29 days	8 / 7 / 7 / 7
30 days	8 / 8 / 7 / 7
31 days	8 / 8 / 8 / 7

Weekly donut model

- The colored portions represent spending in each non-empty period against the full effective category budget, not against a separate per-period budget.
- Render one dimmed remainder for all unused budget. Future or empty periods are absorbed into this remainder instead of receiving their own empty arc.
- The center shows total spent and budget usage percentage when a meaningful percentage exists.
- The legend lists every calendar period in date order with week number, date-only range, spent amount, and entry count.
- A future/empty period shows no percentage label because no spending has happened yet; it still appears in the dated legend with zero amount and zero entries.

Exact 100% requirement

Calculate arc fractions from cent-rounded amounts. Let **B** be the effective budget, **S** the sum of weekly spending, and **R** = **max(B - S, 0)**. When the category is not over budget, the rendered fractions are each non-empty week's **spent / B** plus one remainder of **R / B**, which totals exactly 100%. Do not round each arc independently and accept a drift.

For visible labels, round all but the balancing remainder to the chosen precision and set the final displayed remainder to **100% - the sum of displayed non-empty week values**. This preserves the intended examples: 33.3%, 25%, 10%, and a 31.7% remainder; if whole percentages are used, a 33%, 25%, 10% display must end with 32%.

Over-budget state

When total spending exceeds the effective budget, remove the remainder entirely. Render weekly spending with the same over-budget red treatment used by the existing donut. Never display a negative remainder or a fabricated future-week percentage.

Worked example

Period	Date range	Spent	Entries	Display rule
Week 1	01.04-08.04	400	4	33.3% of 1,200
Week 2	09.04-16.04	300	3	25% of 1,200
Week 3	17.04-22.04	120	2	10% of 1,200
Week 4	23.04-30.04	0	0	No percentage; future/empty
Remainder	All unused budget	380	-	31.7%, dimmed

Phase A - Verify the existing contract and helpers

- Trace the current dashboard summary's ownership, currency, locked-entry, realized-goal, Larder, and stretch-adjustment logic.
- Confirm the existing category-weeks OpenAPI path, schemas, generated client hook, and backend implementation all describe the same response.
- Add or update pure calendar allocation helpers so 28/29/30/31-day months produce the locked distributions and actual date boundaries.
- Keep generated files generated: change the OpenAPI source first, then rerun codegen before consuming changed types.

Phase B - Complete the backend aggregation

- Implement the authenticated category-weeks handler in `export/backend/src/routes/summary.ts`.
- Reuse existing summary query predicates and effective-budget calculations instead of duplicating business rules.
- Aggregate by transaction date into the four server-defined periods; return spent and entriesCount per period.
- Ensure empty/future periods are returned with zero values and valid date-only boundaries.
- Return explicit authorization/validation failures for invalid or unauthorized requests.

Phase C - Add the weekly visual state

- Add `export/frontend/src/components/WeeklyCategoryDonut.tsx` for weekly arc construction, the single dimmed remainder, center labels, and dated legend.
- Keep the component data-driven and avoid client-side transaction fetching.
- Use the selected category's existing color for spent arcs and the existing red over-budget treatment when applicable.
- Make narrow segments and all interactive targets accessible to touch, following the existing donut hit-target approach.

Phase D - Wire Dashboard and interactions

- Add selected category and weekly drill-down state to `export/frontend/src/pages/Dashboard.tsx`.
- Add the 500 ms long-press lifecycle to `export/frontend/src/components/DonutBudgetChart.tsx`, including cancellation and click suppression.
- Start the weekly query only after an eligible category is selected; key it by month, categoryId, and currency.
- Reuse the existing household/personal transition timing and cleanup timers.
- Back clears weekly state; dashboard mount/re-entry starts at the default category donut.

Phase E - Transaction handoff and localization

- Navigate the Show transactions action to the existing home/spending tab at `/`.
- Pass the selected month in the existing full-month loading mechanism and apply the category-name filter.
- Ensure the home list has all records for that month before filtering; do not filter a first-page/first-15 subset.
- Allow the user to clear the category filter and return to the complete month list.
- Add week/date-range labels and back/help/CTA copy to `export/frontend/src/lib/i18n.ts` for supported languages.

Calendar and data checks

- February with 28 days produces 7/7/7/7 and contiguous dates.
- Leap-year February with 29 days produces 8/7/7/7 and contiguous dates.
- April, June, September, and November with 30 days produce 8/8/7/7.
- A 31-day month produces 8/8/8/7.
- The four periods cover the month once, with no gaps or duplicate boundary dates.
- Weekly spent totals reconcile exactly to the category total for the same query rules.
- Effective budget matches the existing dashboard donut, including stretch adjustments.
- Arc fractions sum to exactly 100% in the non-over-budget state after cent rounding and balancing.

Interaction checks

- Long press opens the weekly view only for a positive-budget category with spending.
- Short tap still selects the category exactly as before.
- Center double-tap mode switching remains unchanged.
- Movement, release, and cancellation stop an in-progress hold.
- Uncategorized, recurring-payment, unbudgeted, and empty segments do nothing.
- Back returns to the original donut and removes the weekly state.
- Animation and cleanup behavior matches the household/personal donut sequence.

Display and handoff checks

- Legend periods are date-only and listed in chronological order.
- A future/empty period has no percentage label.
- Unused budget is one dimmed remainder, not four separate dimmed slices.
- The example 400/300/120 against 1,200 displays 33.3%, 25%, 10%, and 31.7% remainder.
- If the category is over budget, the remainder disappears and the existing red treatment is used.
- Show transactions opens the home tab with the full selected month and category-name filter.
- Clearing the filter restores all records for the selected month.
- Re-entering the dashboard starts in its default view.

Build and release checks

- Frontend typecheck passes.
- Backend typecheck/build passes.
- OpenAPI codegen completes after any contract change.
- The shipping diff contains changes only under export/frontend, export/backend, and the required contract/generated sources.
- Push the completed shipping changes to GitHub; do not trigger Vercel or Render deploys.

Definition of done

The feature is ready for the user's manual deployment step only when the weekly API, weekly donut, long-press behavior, exact 100% math, full-month transaction handoff, dashboard reset, and all calendar cases above pass together.

This updated plan supersedes the earlier draft where it conflicts. The single dimmed remainder, exact percentage balancing, date-only labels, no percentage for future/empty periods, full-month handoff, and 1:1 animation reuse are locked requirements.